IdeaTrack Frontend Technical Documentation

Comprehensive architecture and module documentation for the IdeaTrack React frontend, with special focus on User Management, authentication, authorization, profile handling, and reviewer-stage assignment.

PROJECT TYPE
React SPA for enterprise innovation management

BUILD STACK
React 19, TypeScript, Vite, Axios, React Router 7

PREPARED ON
March 15, 2026

1. Project Introduction

IdeaTrack is a role-based internal innovation platform where employees submit ideas, reviewers evaluate them in staged workflows, and administrators govern user access, categories, analytics, and proposal operations.

What the Frontend Does

- Provides a complete browser-based interface for idea submission, review, approval, proposal creation, analytics, and profile management.
- Applies role-aware navigation and route protection so each user sees only the surfaces relevant to their responsibilities.
- Connects to a Spring Boot backend through REST APIs and server-sent events for real-time notifications.

Primary User Groups

- **Employees**: create ideas, track status, manage proposals, and maintain their profiles.
- **Reviewers**: review assigned ideas, collaborate in stage discussions, and submit decisions.
- **Admins**: manage users, categories, reports, overdue reviewers, bulk operations, and system health.
- **Super Admins**: manage the full user base including admins.

Business Problem Solved

The application replaces fragmented idea handling across spreadsheets, email, and manual approvals with a structured digital workflow. It centralizes user access, review assignments, notifications, analytics, and audit-friendly user operations in a single interface.

Role of the User Management Module

User Management is the operational control center for platform access. It allows admins and super admins to create accounts, assign roles, update user details, delete users, and configure reviewer-stage mappings that directly affect the review pipeline.

2. Technology Stack

Area	Implementation	Why It Is Used
UI Framework	React 19.2.0	Component-based composition, modern hook model, and scalable SPA development.
Build Tool	Vite 7 with `@vitejs/plugin-react-swc`	Fast startup, fast HMR, lightweight configuration, and modern TypeScript-friendly development.
Language	TypeScript 5.9	Improves maintainability through static typing for DTOs, props, and API contracts.
Routing	react-router-dom 7.13	Supports route-driven UX, nested layouts, redirects, and role-aware protection.
Global State	React Context	Used selectively for auth, notifications, and toasts without introducing Redux-level complexity.
HTTP Client	Axios 1.13	Centralized interceptors, cleaner request configuration, auth header injection, and consistent error handling.
Forms	Native controlled state plus react-hook-form in selected admin forms	Simple forms stay lightweight; complex forms such as category management benefit from validation and field registration.
Styling	Plain CSS, CSS modules, Bootstrap 5, and inline component styles	Allows quick UI delivery, component-specific encapsulation, and responsive utility support.
Charts	Recharts	Provides dashboard-ready chart primitives for employee, reviewer, and admin analytics.
Icons	lucide-react	Consistent iconography across dashboards, admin actions, profile controls, and notifications.
Auth Handling	JWT in localStorage plus client-side role decoding	Enables stateless route gating and request authorization across the SPA.

Architecture note: the project does not use a dedicated `services` folder. The service layer lives in `src/utils`, which effectively acts as the API and helper layer.

3. System Architecture

Frontend Architecture Pattern

The frontend follows a feature-oriented SPA architecture with route-driven pages, reusable components, and a thin global state layer. Most business data is fetched close to the page or feature component that needs it, while auth, notifications, and toast feedback are elevated into providers.

```
Browser
  -> React App
    -> App.tsx providers
      -> AuthProvider
      -> NotificationProvider
      -> ToastProvider
      -> BrowserRouter
        -> ProtectedRoute
          -> Layout
            -> Feature Pages
              -> Components
                -> utils/* API modules
                  -> Spring Boot backend (REST + SSE)
```

Provider Layer

Auth, notifications, and toast feedback are initialized once near the application root and reused throughout the route tree.

Route Layer

Routes separate public pages from authenticated pages and apply role restrictions through a dedicated guard component.

Feature Layer

Pages assemble feature-specific components and call API modules directly rather than relying on a centralized data store.

4. Folder Structure

```
src/
  App.tsx                  Application routes and provider composition
  main.tsx                 React entry point
  components/             Reusable UI and feature components
    admin/                Category and bulk admin tools
    auth/                 Route protection and auth styling
    ConfirmationModal/    Shared confirmation dialog
    Hierarchy/            Idea hierarchy and profile modal views
    idea/                 Idea cards, idea details, idea timeline components
    layout/               Sidebar, header, shell layout
    notifications/        Bell, list items, notification UI
    ProfileHub/           Profile tabs, type mapping, presentation logic
    proposals/            Proposal forms and proposal editing controls
    reviewer/             Reviewer decision and discussion components
    ReviewerStageAssignment/
    Toast/                Toast renderer
    user/                 User cards, registration modal, user edit modal
  context/                Global providers for notifications and toast state
  hooks/                  Small shared hooks for error handling and toasts
  pages/                  Route-level screens grouped by domain
    admin/
    analytics/
    auth/
    employee/
    idea/
    landing/
    notifications/
    profile/
    proposal/
    reviewer/
  utils/                  API clients, auth helpers, storage, interceptors, shared types
  assets/                 Static source assets
```

Where User Management Lives

- **Admin user operations**: `src/pages/admin` plus `src/components/user`.
- **Access control**: `src/utils/authContext.tsx` and `src/components/auth/ProtectedRoute.tsx`.
- **User data APIs**: `src/utils/adminApi.ts`, `src/utils/profileApi.ts`, and `src/utils/reviewerAssignmentApi.ts`.
- **Self-service account management**: `src/pages/profile/ProfileHub.tsx` plus the `ProfileHub` component folder.

Why This Structure Works

This structure keeps route ownership clear while still allowing cross-cutting behavior to be shared. Pages remain responsible for feature orchestration, components focus on rendering and interaction, and utilities isolate backend communication and storage logic.

5. User Management Module Architecture

The User Management module spans account provisioning, user editing, profile maintenance, role governance, and reviewer assignment. It is not a single page; it is a coordinated set of admin, profile, and access-control features.

Core Components

Component	File Location	Purpose	Props
AdminConsole	src/pages/admin/AdminConsole.tsx	Admin-facing user management for reviewers and employees, including stage assignment tab.	None
SuperAdminConsole	src/pages/admin/SuperAdminConsole.tsx	Full user management including admin accounts.	None
UserCard	src/components/user/UserCard.tsx	Renders a user summary card in the management list.	`user`, `onMa`
RegisterUserModal	src/components/user/RegisterUserModal.tsx	Creates new users and displays the generated temporary password after success.	`open`, `onCl` `onSubmit`, `allowedRo`
UserModal	src/components/user/UserModal.tsx	Edits user profile-like fields, role, department, and deletes the selected user.	`open`, `user` `onClose`, `onChanged` `allowRole`
ReviewerStageAssignment	src/pages/reviewer/ReviewerStageAssignment.tsx	Switches between reviewer assignment creation and current assignment display.	None

Component	File Location	Purpose	Props
ReviewerStageAssignment_Creation_Module	src/components/ReviewerStageAssignment/ReviewerStageAssignment_Creation_Module.tsx	Assigns a reviewer to a category stage after selecting department, category, stage, and reviewer.	None
ReviewerStageAssignment_Display_Module	src/components/ReviewerStageAssignment/ReviewerStageAssignment_Display_Module.tsx	Shows reviewer-stage mappings and allows removals.	None
ProfileHub	src/pages/profile/ProfileHub.tsx	Self-service account area for profile editing, activity history, achievements, and security.	None
OverviewTab	src/components/ProfileHub/OverviewTab.tsx	Validates and updates editable profile fields such as name, phone, and bio.	`user`, `isEd` `onProfile` `onEditing`
SecurityTab	src/components/ProfileHub/SecurityTab.tsx	Handles password changes and soft deletion of the current user profile.	`onPasswor`
ProtectedRoute	src/components/auth/ProtectedRoute.tsx	Guards user-management routes and all other protected routes based on authentication and role allow-lists.	`allow`, `chi`

User Management Workflow

1. Admin navigates to **/admin/users** or super admin navigates to **/super-admin**.
2. The page loads users from `/api/admin/all` and filters visible roles according to console type.
3. Tabs segment the list by role. Search narrows results on name, email, user ID, and department.
4. Creating a user opens RegisterUserModal, which loads departments, validates basic input, and posts to `/api/admin/create`.
5. On success, the UI shows a temporary password popup and refreshes the user list.
6. Selecting a user opens UserModal, which computes a field-level diff and submits only changed values to `/api/admin/{id}`.
7. Deleting a user goes through ConfirmationModal, then calls `DELETE /api/admin/{id}`, refreshes the list, and emits toast feedback.

8. The Stage Assign tab lets admins bind reviewers to department category stages, which directly influences reviewer workload and downstream review flow.

Role and Permission Model

The frontend enforces coarse-grained access via route guards and fine-grained operational limits through page configuration. For example, AdminConsole only allows creation and editing of reviewers and employees, while SuperAdminConsole additionally exposes admin-level role handling.

EMPLOYEE REVIEWER ADMIN SUPERADMIN

6. State Management

Where State Is Stored

- **AuthProvider**: JWT token, decoded payload, roles, login, and logout.
- **NotificationProvider**: live notification list, unread count, connection status, and singleton SSE connection lifecycle.
- **ToastProvider**: transient UI feedback messages.
- **Feature-local state**: user lists, forms, active tabs, modals, and loading flags remain inside pages and components.

Global State vs Local State

Global	Local
Authentication, notification stream, toast queue	Admin console tab, selected user, search term, modal open state, profile edit mode

Data Flow Pattern in User Management

```
Admin action
  -> component event handler
    -> utils/adminApi.ts request
      -> backend response
        -> local state refresh or modal close
          -> toast feedback
            -> rerendered user list
```

The project intentionally avoids Redux or Zustand. This keeps the state model simple but pushes more responsibility onto page components to refetch and synchronize their data explicitly after mutations.

7. API Integration

Axios Configuration

- `src/utils/restApi.ts` defines a shared Axios instance with base URL `http://localhost:8091`.
- Request interceptor adds `Authorization: Bearer <jwt-token>` when present.
- Response interceptors log 401 and 403 cases; a separate app-level interceptor translates many backend errors into user-facing toasts.

User Management Endpoints

Endpoint	Method	Purpose
/api/admin/all	GET	Fetches all users for admin consoles.
/api/admin/create	POST	Creates a user and returns temp password data.
/api/admin/{id}	PUT	Updates user fields such as role, contact data, and profile URL.
/api/admin/{id}	DELETE	Deletes a user account.
/api/profile/departments	GET	Supplies department options to admin and signup forms.
/api/users/{id}/status	PATCH	Reserved for status toggling, though not a major UI flow yet.
/api/profile/me	GET / PUT / DELETE	Self-service profile retrieval, update, and soft delete.
/api/profile/me/password	PUT	Password change flow.
/api/profile/me/profile-photo	POST / DELETE	Profile image upload and removal.
/api/reviewerAssignment/*	GET / POST / DELETE	Reviewer-stage assignment configuration.

Request Lifecycle Example

```
Click "Create User"
  -> RegisterUserModal submit()
    -> createUser(payload)
      -> POST /api/admin/create
        -> backend returns created user + tempPassword
          -> modal shows temp password
            -> parent reloads listUsers()
              -> UI updates user cards
```

Error Handling and Loading States

- Most feature components use local `loading`, `saving`, or `submitting` flags.
- Axios error interception standardizes many server responses into readable toast messages.
- Several admin operations still rely on local `try/catch` plus console logging for secondary diagnostics.

8. Authentication & Authorization

Login Flow

1. User submits email and password on the login page.
2. `AuthProvider.login()` posts credentials to `/api/auth/login`.
3. The backend returns a JWT token.
4. The token is saved to localStorage under `jwt-token`.
5. The token is decoded client-side to extract roles.
6. The user is redirected to admin, reviewer, or employee landing routes based on highest role.

Protected Routes

`ProtectedRoute` blocks unauthenticated users and optionally checks whether at least one of the user's roles appears in the route's `allow` list.

How User Management Respects Access Control

- **/admin/users** is limited to ADMIN and SUPERADMIN.
- **/super-admin** is limited to SUPERADMIN only.
- AdminConsole restricts role creation and editing to reviewer and employee accounts.
- SuperAdminConsole expands this to include admin accounts.

> **Security caution:** role checks are client-side UX gates and must still be enforced by the backend. The frontend correctly assumes the backend remains the source of authorization truth.

9. User Journey

1. User opens the application and lands on a public marketing-style landing page.
2. User signs up or logs in.
3. After login, routing redirects based on role: employee dashboard, reviewer dashboard, or admin dashboard.
4. Employees create ideas, view idea details, and eventually convert accepted ideas into proposals.
5. Reviewers work from their dashboard, inspect assigned ideas, and submit decisions or stage discussions.
6. Admins manage platform structure through user management, category management, bulk operations, reports, and health views.
7. Users maintain personal details in Profile Hub, including password and profile photo.
8. Notifications appear in real time through SSE and remain available in a paginated activity page.

User Management Journey

```
Admin Dashboard
  -> User Management
    -> role tab selection
      -> user list fetch
        -> create or manage selected user
          -> reviewer assignment if needed
            -> refreshed state + toast confirmation
```

10. Performance Optimization

Technique	Current Status	Observation
Lazy loading	Not implemented	All route components are imported eagerly in App.tsx.
Code splitting	Limited	Relies on Vite bundling, but no route-level `lazy()` strategy is used.
Memoization	Selective	`useMemo` is used in admin filtering, payload diffing, and toast helper creation.
useCallback	Selective	Used where repeated handlers benefit from stability, especially around data loading.
Pagination	Partial	Notifications and some reviewer discussion endpoints support paging; admin user lists do not yet page.
Virtualization	Not present	Large lists still render directly in the DOM.

The current application is performant enough for moderate datasets, but user-management scalability would improve with server-side pagination, route-level lazy loading, and a cache layer such as React Query.

11. Security

- **Token protection**: JWT is stored in localStorage and automatically attached to API calls. This is simple but carries XSS exposure if the application were ever compromised.
- **XSS mitigation**: the frontend mostly renders plain strings and does not rely on unsafe HTML insertion.
- **Input validation**: profile and security forms include client-side validation; create-user flow uses minimal client checks and leans on backend validation.
- **Secure API usage**: auth headers are centralized, and protected routes reduce accidental exposure of privileged screens.
- **Role-based security**: user-management pages are visible only to authorized roles, while backend endpoints should remain the final enforcement layer.

Security Improvement Opportunities

- Move API base URL to environment variables.
- Add proactive token expiration handling instead of waiting for 401 failures.
- Harden duplicate-email validation before user creation to improve UX.

12. Error Handling

Implemented Handling

- Global Axios interceptor maps server errors into toast messages.
- Feature components catch specific failures such as failed profile loads, failed user updates, or failed assignment removals.
- Loading spinners and empty states exist on dashboards, notifications, assignments, and profile screens.

User Management Edge Cases Covered

- Department fetch failure when creating users.
- Save-without-changes prevention in UserModal.
- Password policy validation before changing credentials.
- Confirmation before destructive actions such as deleting users or profiles.

Edge Cases Still Mostly Backend-Driven

- Duplicate email on user creation.
- Permission denied after stale client role state.
- Expired JWT during session.
- Malformed or missing notification metadata.

13. How to Explain the User Management Module in an Interview

2-Minute Version

I built a role-based user-management flow inside a React and TypeScript SPA. Admins and super admins can create users, assign roles, update user details, delete accounts, and configure reviewer-stage assignments that drive the review workflow. Access is controlled through JWT-based auth, route guards, and role-aware UI composition. The module uses Axios for API calls, React Context for auth and notifications, and local component state for operational screens.

5-Minute Deep Explanation

The system separates concerns cleanly. App.tsx composes global providers and route protection. AuthProvider decodes JWT roles and exposes them through a hook, so navigation and protected routes can respond immediately to the user's privileges. The admin user-management pages then specialize behavior further: AdminConsole only manages reviewer and employee accounts, while SuperAdminConsole can also manage admins. User creation and editing are modal-driven. RegisterUserModal loads departments, submits account creation, and surfaces the generated temporary password. UserModal computes a diff payload so unchanged fields are not resent. In parallel, the profile subsystem handles self-service updates, password changes, photo management, and activity history. ReviewerStageAssignment extends user management into workflow governance, because who reviews what stage is effectively part of access and responsibility management.

Key Technical Highlights

- JWT decoding without an external auth SDK.
- Context-based auth, notifications, and toasts instead of heavyweight global state.
- Role-aware route protection and role-aware console behavior.
- Reusable modal architecture for create, edit, and confirm-delete flows.
- Singleton SSE notification stream to avoid duplicate real-time connections.

Challenges Solved

- Preventing duplicate SSE connections across multiple notification consumers.
- Supporting both admin and super admin workflows without duplicating all component logic.
- Handling profile data, auth state, and user-admin operations without a large external state library.

Scalability Considerations

- Add server-side pagination and filtering to admin user lists.
- Introduce React Query or a similar cache layer for mutation invalidation and consistent loading states.
- Split route bundles lazily for faster first load.

14. Client-Friendly Explanation

IdeaTrack is a web application that helps an organization collect ideas, review them properly, and turn strong ideas into formal proposals. Different people see different tools depending on their role. Employees submit ideas, reviewers assess them, and administrators manage users, categories, reporting, and system setup.

The User Management area lets administrators add new users, assign what kind of access they should have, update their information, and organize which reviewers handle which review stages. That makes the platform easier to operate and keeps responsibilities clear.

From a business perspective, the architecture is scalable because the application already separates authentication, user operations, profile management, notifications, and analytics into distinct modules. That means new roles, new screens, or deeper reporting can be added without redesigning the whole product.

15. Conclusion

IdeaTrack's frontend is a well-structured React SPA built around role-based operations and a clear provider-driven shell. Its strongest architectural qualities are the separation between route orchestration, reusable feature components, and API utilities, plus a focused approach to global state.

The User Management module is technically significant because it connects access control, account lifecycle management, self-service profile operations, and reviewer assignment into one coherent operational capability. For an interview, it demonstrates practical frontend architecture, business workflow modeling, and real-world admin UX design. For a client, it demonstrates control, governance, and scalability.

> This document is based on the current implementation in the workspace, including routes, components, contexts, hooks, and API utility modules.